MEKDELA AMBA UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATICS

DEPARTMENT OF COMPUTER SCIENSE

Course Title:Artificial Intellegence (Ai)

Project Title:Fake News Detection System

Group Two(2) Project Of Fake News Detection System Final Reports

No	Name	Id No
1	Beyene Demeke	Mau/1602191
2	Simegnaw Munye	Mau/ 1601098
3	Aklilu Ayika	Mau/1600090
4	Yeamlaksira Adane	Mau/1601300

Submited To Misgie(Msc) **Submission Date 7/08/2018 E.C**

Table of Contents

Abstract

This project builds a fake news detection system using natural language processing and machine learning methods. The system uses the LIAR dataset, which contains short political statements labeled on a credibility scale. We preprocess news text, train three classification models, measure performance with standard metrics, and deploy a user-friendly interface using Streamlit.

The final system includes a baseline model, an advanced ensemble model, and a neural sequence model. Evaluation results show that while the baseline and ensemble models perform similarly, the LSTM model provides a stronger balance of precision and recall for binary fake/real classification with noisy political text.

Introduction

Fake news detection is a critical task in today's information landscape. The rapid spread of misinformation online has motivated machine learning researchers to build systems that classify news statements as real or fake.

This project implements a fake news classifier that:

- **uses text-based features from short statements,**
- **applies data cleaning and preprocessing,**
- **trains both baseline and advanced supervised models,**
- **evaluates each model using accuracy, precision, recall, and F1-score,**
- **deploys the model through a simple Streamlit interface.**

The objective is to compare models critically and create a deployable application for demonstration.

1. Dataset and Citation

1.1 Dataset Source

The project uses the LIAR dataset, a well-known dataset for fake news and political statement classification. The dataset is cited as:

William Yang Wang. "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. arXiv:1705.00648, 2017.

The dataset is available from the University of California, Santa Barbara:

https://www.cs.ucsb.edu/~william/data/liar_dataset.zip

1.2 Dataset Description

The dataset uses tab-separated values (TSV) files with the following columns

1. Statement ID

2. Label (truthfulness category)

3. Statement text

4. Subject(s)

5. Speaker

6. Speaker job title

7. State information

8. Party affiliation

9-13. Credit history counts (barely true, false, half true, mostly true, pants on fire)

14. Context

Only the statement text and label are used in the current classification models.

1.3 Dataset Size

- **The dataset splits are:**
- **Training set: 10,240 samples**
- **Validation set: 1,284 samples**
- **Test set: 1,267 samples**
- **This size satisfies the project requirement of a minimum of 1,00**

2. Data Cleaning and Preprocessing

2.1 Data Cleaning

The preprocessing pipeline includes:

- ✓ **removing punctuation,**
- ✓ **converting text to lowercase,**
- ✓ **removing English stop words,**
- ✓ **applying lemmatization.**

This pipeline reduces lexical noise and standardizes input text for vectorization and sequence modeling.

2.2 Label Mapping

The original dataset contains multiple truth labels. For binary classification, labels are mapped as follows:

- **`true`, `mostly-true` → Real (1)**
- **all other labels → Fake (0)**

This mapping enables the system to perform a binary real/fake classification task.

2.3 Feature Extraction

Two feature representations are used:

TF-IDF vector for the Logistic Regression and Random Forest models.

Tokenized padded sequences for the LSTM neural model.

TF-IDF captures term importance across the dataset, while sequence tokenization preserves word order for the LST

3. Modeling Approach

This project implements three models with different levels of complexity:

1. Baseline: Logistic Regression

2. Advanced:Random Forest

3. Neural Sequence Model: LSTM

3.1 Model Justification

Logistic Regression is a strong text baseline that is fast, interpretable, and appropriate for sparse TF-IDF features.Random Forest provides a more advanced ensemble approach that can learn non-linear decision boundaries and handle noisy feature interactions.

LSTM is chosen as an advanced neural model because it can capture sequential dependencies in the text and learn patterns that are not available in simple bag-of-words representations.

3.2 Implementation Details

Logistic Regression and Random Forest use TF-IDF vectors generated from the cleaned statements.

The LSTM model uses a vocabulary limited to 5,000 tokens, an embedding layer, a single LSTM layer with dropout, and a sigmoid output for binary classification.

The LSTM is trained for 5 epochs with a batch size of 64 and uses a validation split drawn from the combined training set.

4. Baseline Model: Logistic Regression

4.1 Rationale

Logistic Regression is an appropriate baseline because it is commonly used in NLP classification tasks and often performs well with TF-IDF representations. It offers a meaningful benchmark against which more complex models can be evaluated.

4.2 Strengths

- **Fast training and inference**
- **Interpretable coefficients**
- **Works well with high-dimensional sparse features**

4.3 Limitations

Assumes a linear relationship between input features and the log-odds output.

May underperform when decision boundaries are highly non-linear.

5. Advanced Model: Random Forest

5.1 Rationale

Random Forest is an ensemble classifier that improves on single-tree approaches by averaging multiple decision trees. It is robust against overfitting and can capture complex feature interactions.

5.2 Strengths

Handles non-linearity better than linear models

Reduces variance through aggregation of many trees

Good default choice for many tabular and feature-based problems

5.3 Limitations

May require more computation and memory than logistic regression

Less interpretable than a single linear model

6. Neural Model: LSTM

6.1 Rationale

The LSTM model is chosen to capture sequential patterns and context within the statement text. Unlike TF-IDF, the LSTM preserves word order and can model dependencies between words.

6.2 Architecture

The implemented architecture includes:

Embedding layer with vocabulary size 5,000 and embedding dimension 128

LSTM layer with 64 units and dropout

Dense output layer with sigmoid activation

6.3 Strengths

Captures sequential and contextual information

Learns richer text representations than bag-of-words

6.4 Limitations

- **Longer training time.**
- **May require more data to generalize well.**
- **Sensitive to hyperparameters and input sequence length.**

7. Evaluation Methodology

7.1 Metrics

The following metrics are used to evaluate classification performance:

- ✓ **Accuracy: overall correctness of predicted labels**
- ✓ **Precision: fraction of predicted real labels that were correct**
- ✓ **Recall: fraction of actual real samples that were correctly predicted**
- ✓ **F1-score: harmonic mean of precision and recall**

7.2 Validation Strategy

The project uses the provided dataset splits:

- **Training set for model learning**
- **Validation set for neural model training and tuning**
- **Test set for final evaluation**

This approach satisfies the project requirement of using train/test split or cross-validation.

7.3 Evaluation Output

These metrics were obtained by running the project training script on the provided dataset splits.

Logistic Regression Results:-

- **Accuracy: 0.6464**
- **Precision: 0.5025**
- **Recall: 0.2227**
- **F1-score: 0.3086**
- **Random Forest Results**
- **Accuracy: 0.6448**
- **Precision: 0.4974**
- **Recall: 0.2116**
- **F1-score: 0.2969**

LSTM Results:-

- ✓ **Accuracy: 0.5912**
- ✓ **Precision: 0.4214**
- ✓ **Recall: 0.4120**
- ✓ **F1-score: 0.4167**

8. Results Discussion

8.1 Model Comparison

Logistic Regression achieved the highest overall accuracy among the three models, but its recall was low. This indicates the model is more conservative about labeling statements as real.

Forest performed very similarly to Logistic Regression, with slightly lower accuracy and F1-score.

LSTM achieved the best F1-score among the three, indicating a better balance between precision and recall even though its overall accuracy was lower.

8.2 Interpretation

The dataset appears to be challenging for binary fake/real classification, especially when labels are derived from a multi-class credibility scale.

TF-IDF-based models are strong baselines for this type of task, likely because the statements are short and contain informative keywords.

The LSTM model demonstrates that sequential modeling can improve balance between precision and recall, but it may require more advanced tuning or larger datasets for improved accuracy.

8.3 Limitations

The binary label mapping reduces the original multi-class nuance of the dataset.

The current models use only statement text; speaker metadata, context, and party affiliation are not used.

The LSTM model was trained for only five epochs; additional training or hyperparameter tuning could improve performance.

9. Deployment

9.1 Interface Design

The project includes a deployment interface using Streamli.

The Streamlit app allows users to:

- **inter a news article or statement,**
- **select between Logistic Regression, Random Forest, or LSTM models,**
- **receive a prediction of Real or Fake,**
- **view a confidence estimate for the selected model.**

9.2 Deployment Workflow

The deployment workflow is:

1. Load trained models (`lr_model.pkl`, `rf_model.pkl`, `lstm_model.h5`).

2. Preprocess user input using the same cleaning pipeline as training.

3. Vectorize or tokenize input text based on model type.

4. Make a prediction and display the result in the Streamlit UI.

9.3 Usage

To run the app:

- ✓ **Install dependencies from `requirements.txt`**
- ✓ **Start the app with `streamlit run app.py`**

10. Conclusions

This project successfully implements a fake news detection system that spans dataset preparation, model training, evaluation, and deployment. Key conclusions include:

- ❖ **The LIAR dataset provides a solid real-world benchmark with more than 12,000 training samples.**
- ❖ **Logistic Regression and Random Forest are strong baseline methods for text classification using TF-IDF.**
- ❖ **LSTM can improve balance between precision and recall, even when overall accuracy is lower.**
- ❖ **A simple Streamlit deployment makes the system accessible and demonstrable.**

10.1 Future Work

Future improvements could include:

- ➢ **using the full multi-class labels instead of binary mapping,**
- ➢ **incorporating metadata features such as speaker, party, and context,**
- ➢ **using more advanced text architectures such as Transformers,**
- ➢ **tuning hyperparameters with cross-validation,**
- ➢ **adding explainability features to show which words influenced predictions**

11. Appendix

11.1 Dataset Details

The dataset's `test.tsv` file contains 1,267 samples. The validation set contains 1,284 samples and is used during LSTM training to monitor validation loss and accuracy.

11.2 Model Files

Trained model files included in the repository:

`lr_model.pkl`

`rf_model.pkl`

`lstm_model.h5`

`tfidf.pkl`

`tokenizer.pkl`

11.3 Notes on Output

The evaluation metrics listed in Section 10.3 are based on direct model output from the provided `model_training.py` script.

12. References

NLTK documentation: https://www.nltk.org/

Scikit-learn documentation: https://scikit-learn.org/

Streamlit documentation: https://streamlit.io/

TensorFlow Keras documentation: https://www.tensorflow.org/

Wang, W. Y. (2017). "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. arXiv:1705.00648.